

Plan de Pruebas y Diseño de Escenarios — TicketResolve

Curso: Testing para Software — Postgrado en Diseño y Desarrollo de Software, Universidad Galileo

Sistema bajo prueba (SUT): TicketResolve — Plataforma de gestión de incidentes y tickets **Autores:**

Pablo Pineda (21010478) · Christian Martínez (22001222) **Fecha:** 2026-05-27 **Versión:** 2.1

v2.0 — Cambios por retroalimentación recibida. Elabora el **contenido de cada nivel de prueba con sus herramientas concretas** (API → Postman/Newman, UI → Cypress/Playwright, etc.) y amplía **rendimiento, estrés, seguridad, UAT, riesgos y escenarios**. **v2.1 — Incorpora KPIs operativos (MTTD/MTTR), atributos de calidad cloud, clasificación de severidad de defectos (S1–S4), cronograma de ejecución, herramientas de gestión/trazabilidad y casos de RBAC y resiliencia**, adaptados a la arquitectura serverless real (Lambda/DynamoDB/SQS), no a un modelo de instancias/microservicios.

Tabla de contenido

1. Contexto y enfoque de calidad
 2. Plan de pruebas (estructura ISTQB)
 3. Niveles y capas de prueba — contenido y herramientas
 4. Tipos de prueba especializados
 5. Requerimientos a probar
 6. Selección de técnicas por requerimiento
 7. Técnicas de caja negra
 8. Técnicas de caja blanca
 9. Diseño de escenarios de prueba detallados
 10. Especificación por comportamiento (BDD / Gherkin)
 11. Pruebas no funcionales (rendimiento, estrés, seguridad)
 12. Pruebas de aceptación de usuario (UAT)
 13. Gestión de riesgos
 14. Matriz de trazabilidad (RTM)
 15. Supuestos y datos de prueba
 16. Clasificación de severidad y prioridad de defectos
 17. Cronograma de ejecución
 18. Anexos — artefactos de prueba ejecutables
-

1. Contexto y enfoque de calidad

1.1 La calidad es transversal al SDLC

Siguiendo el principio del curso de que *la calidad no es una etapa final sino transversal*, este plan distribuye actividades de prueba a lo largo del ciclo de vida de TicketResolve:

Etapa SDLC	Actividad de calidad en TicketResolve
Análisis	Revisión de las User Stories (US-01...US-06) buscando ambigüedades (ej. SLA exacto por severidad no estaba definido → ver supuestos).
Diseño	Creación de este plan de pruebas y los escenarios derivados de los criterios de aceptación.
Desarrollo	Pruebas unitarias por cada Lambda (<code>api-tickets</code> , <code>webhook-ingesta</code> , <code>escalamiento-worker</code> , etc.) — primer filtro de calidad.
Pruebas	Funcionales, integración Lambda→DynamoDB→SQS→SNS, seguridad y rendimiento.
Implementación	UAT con el Gerente de Operaciones y el Administrador de Plataforma usando datos reales.
Mantenimiento	Pruebas de regresión y <i>sanity</i> al replicar incidentes reportados.

1.2 Justificación económica (costo de los defectos)

El curso establece la escalabilidad del costo de un defecto: **1x** en requerimientos → **30x** en producción. En un sistema de incidentes esto es crítico: un defecto en el **motor de escalamiento** que llegue a producción no solo cuesta 30x corregirlo, sino que puede dejar un **P0 sin atender** (incumplimiento de SLA con impacto operativo real). Por eso este plan invierte fuerte en **revisiones estáticas** y **pruebas unitarias** tempranas sobre la lógica de deduplicación y escalamiento, que son las de mayor riesgo.

1.3 Niveles de acceso aplicados al SUT

Nivel	Aplicación en TicketResolve
Caja negra	Pruebas funcionales contra la API HTTP (API Gateway) y la UI usando solo contratos request/response. Sin mirar el código.
Caja blanca	Verificación lógica de <code>webhook-ingesta</code> (hashing/dedup) y <code>escalamiento-worker</code> (conditional writes), con acceso a código, DynamoDB y CloudWatch Logs.
Caja gris	Pruebas de integración: se conocen endpoints y el esquema DynamoDB (single-table, GSIs) pero no el detalle interno de cada función.

1.4 Atributos de calidad cloud y KPIs operativos

Por ser un sistema *cloud-native* y de gestión de incidentes (ITSM), además de lo funcional se validan estos atributos — **adaptados a la arquitectura serverless real** (Lambda/DynamoDB/SQS/SNS),

donde el escalado y la alta disponibilidad los aportan los servicios administrados, no instancias que haya que aprovisionar:

Atributo	Qué significa en TicketResolve	Cómo se verifica
Alta disponibilidad / resiliencia	Servicios administrados multi-AZ; ningún webhook se pierde ante fallo transitorio	SQS + DLQ y reintentos; el worker reintentará y, tras N fallos, el mensaje cae a DLQ (TC-12)
Escalabilidad elástica	Concurrencia automática de Lambda y capacidad on-demand de DynamoDB	Prueba de estrés (§11.2): la ráfaga escala sin aprovisionar nada
Seguridad cloud	RBAC vía JWT del IdP + IAM least-privilege; cifrado en tránsito (TLS) y en reposo (SSE en S3/DynamoDB)	TC-11 (RBAC) + análisis estático tfsec/Checkov (§11.3)
Idempotencia	Reprocesar un mensaje no duplica tickets ni notificaciones	Conditional writes (§8.2) + reentrega de SQS

KPIs operativos del negocio que QA valida que el sistema calcule correctamente:

- **MTTD (Mean Time To Detect):** tiempo desde que ocurre el fallo hasta que se crea el ticket/incidente. Relevante para la ingesta por webhook (F-2).
- **MTTR (Mean Time To Resolve):** tiempo desde la apertura hasta `RESOLVED`. Alimenta el reporte mensual (US-06).
- **% de cumplimiento de SLA:** tickets resueltos dentro del plazo vs. total. QA verifica que el congelamiento del cronómetro al cerrar (TC-07) no distorsione la métrica.

Estos KPIs no son solo de negocio: son **datos de prueba verificables** — un escenario puede afirmar "el MTTR reportado coincide con la diferencia real de timestamps en la auditoría de DynamoDB".

2. Plan de pruebas (estructura ISTQB)

2.1 Objetivos

1. Verificar que TicketResolve cumple los criterios de aceptación de US-01 a US-06 y las 3 funcionalidades específicas del dominio.
2. Dar una **visión objetiva** de la madurez del sistema antes de la defensa final (no solo "encontrar fallos").
3. Validar atributos no funcionales críticos: dashboard **<500ms**, ingesta de **500 webhooks**, deduplicación correcta en ventana de **5 min** y que **ningún P0 quede sin escalar tras su SLA**.

2.2 Alcance — qué se prueba en cada capa

El alcance no se define solo como "qué funcionalidades", sino **por capa de prueba**, indicando qué entra y qué no en cada nivel (detalle de ejecución en §3):

Capa	Dentro del alcance (IN)	Fuera del alcance (OUT)
Unitaria	Lógica pura de cada Lambda: cálculo de hash, cálculo de SLA, parseo de payloads	—
API / Servicios	Endpoints de API Gateway: contratos, status codes, validaciones, autenticación JWT, tiempos de respuesta	Lógica interna del IdP corporativo
Integración	Cadena Lambda→DynamoDB→S3→SQS→SNS; escritura/lectura real en staging	Aprovisionamiento Terraform (es del curso de IaC)
UI	Portal de autoservicio (M1), dashboard de ingenieros (M2), consola de diagnóstico (M3)	Compatibilidad con navegadores legacy (IE11)
E2E	Flujos completos actor-a-actor (María reporta → Carlos resuelve → email)	Comunicación de voz
No funcional	Rendimiento, carga, estrés, seguridad	Pruebas de penetración formales por tercero
UAT	Validación de negocio con Admin y Gerente	Capacitación de usuarios finales

2.3 Entornos

Entorno	Uso	Equivalencia (clasificación del curso)
Local / mocks	Pruebas unitarias (DynamoDB Local, mocks de SQS/SNS)	Pruebas internas
Staging (AWS Free Tier)	Integración, API, UI, E2E sobre infra real	Alfa — entorno controlado, equipo presente
Pre-producción / piloto	Uso con un grupo reducido de ingenieros reales	Beta — entorno de usuario, sin equipo presente
Demo / UAT	Validación de negocio con actores clave y datos reales	UAT — "¿sirve para el negocio?"

2.4 Criterios de aceptación del plan (entrada y salida)

Entrada (Entry):

- Build desplegado en staging y *smoke test* en verde.
- User Stories con criterios de aceptación revisados (revisión estática completada).
- Datos de prueba semilla cargados en DynamoDB y colección Postman/entorno configurados.

Salida (Exit):

- 100% de los escenarios P0 (US-01, US-02, US-03) en estado **Passed**.
- $\geq 95\%$ de los escenarios totales **Passed**; 0 defectos abiertos de severidad alta.
- No funcionales dentro de umbral: dashboard p95 < 500ms; 0 duplicados escapados en la prueba de 500 webhooks; 0 P0 sin escalar tras SLA.

2.5 Entregables

- Este plan de pruebas.
- Colección Postman + entorno + scripts de aserción versionados en el repo.
- Suites de UI (Cypress/Playwright) y de carga (k6) versionadas.
- Casos/escenarios detallados (§9) y especificaciones BDD (§10).
- Matriz de trazabilidad (§14).
- Reporte de ejecución con estados (Passed / Failed / Skip-NA / Blocked) + evidencia.
- Reporte de defectos y métricas de cobertura.

2.6 Roles

Rol	Responsabilidad de calidad
Desarrollador	Pruebas unitarias (caja blanca) de su Lambda.
QA / Tester	Diseño y ejecución de API, UI, E2E, no funcionales.
Administrador de Plataforma	UAT de configuración (matriz de escalamiento, guardias).
Gerente de Operaciones	UAT de reportes mensuales.
Equipo completo	Revisiones técnicas / inspecciones (static testing) de requerimientos y diseño.

3. Niveles y capas de prueba — contenido y herramientas

Esta sección responde directamente a la retroalimentación: **qué se prueba en cada nivel y con qué herramienta concreta**. La pirámide va de muchas pruebas baratas y rápidas (unitarias) a pocas pruebas caras y lentas (E2E).

3.1 Mapa general



Nivel	Qué se prueba	Herramienta	Responsable
Unitarias	Lógica interna de un Lambda en aislamiento	Jest (Node) / pytest (Py) + <code>moto</code> , DynamoDB Local	Desarrollador
API / Servicios	Endpoints HTTP: contrato, status, validación, auth, tiempos	Postman (colección) + Newman en CI	QA
Integración	Cadena entre servicios AWS (datos que fluyen y persisten)	Postman + AWS CLI / LocalStack	QA
UI	Portales M1/M2/M3: render, interacción, accesibilidad	Cypress o Playwright	QA
E2E	Flujo completo de negocio actor-a-actor	Playwright (UI) + Postman (verificación backend)	QA

3.2 Pruebas unitarias (caja blanca)

Qué: funciones puras y deterministas, aisladas de AWS con mocks.

Unidad	Caso
<code>calcular_hash(componente, mensaje)</code>	mismo input → mismo hash; inputs distintos → hashes distintos
<code>calcular_sla(severidad)</code>	P0→15min, P1→2h, P2→24h
<code>parsear_webhook(payload)</code>	payload malformado → error controlado, no excepción no manejada

Herramienta: `pytest` + `moto` (mock de DynamoDB/SQS/SNS) o `jest` + `aws-sdk-client-mock`.
 Objetivo de cobertura: **branch coverage ≥ 80%** en `webhook-ingesta` y `escalamiento-worker` (ver §8).

3.3 Pruebas de API / Servicios — Postman + Newman

Qué: se valida el contrato HTTP expuesto por API Gateway sin tocar la UI ni el código (caja negra). Es la capa donde se ejercitan la mayoría de los requerimientos funcionales del backend.

Estructura de la colección Postman:

```

Colección: TicketResolve API (v1)
├── 00 · Auth
│   └── GET token (IdP mock) → guarda {{jwt}}
├── 01 · Tickets
│   ├── POST Crear ticket → 201, guarda {{ticketId}}, t < 2000ms
│   ├── GET Listar por ingeniero → 200, orden por severidad
│   ├── PATCH Cambiar estado → 200, transición válida
│   └── POST Crear ticket SIN token → 401 (caso negativo)
├── 02 · Incidents (Webhook)
│   ├── POST Alerta nueva → 201, crea padre
│   └── POST Alerta duplicada → 200, crea hijo bajo el padre
├── 03 · Reports
│   └── POST Generar reporte mensual → 202 "en proceso"
└── 99 · Negativos / seguridad
    ├── POST Adjunto > 25MB → 413
    └── PATCH CLOSED → IN_PROGRESS → 409 (transición inválida)
  
```

Ejemplo de aserciones (pestaña Tests de Postman) — POST /api/v1/tickets :

```

pm.test("201 Created", () => pm.response.to.have.status(201));
pm.test("Responde en < 2s (US-01)", () =>
  pm.expect(pm.response.responseTime).to.be.below(2000));
pm.test("ID con formato TKT-####", () => {
  const b = pm.response.json();
  pm.expect(b.ticketId).to.match(/^TKT-\d+$/);
  pm.collectionVariables.set("ticketId", b.ticketId); // encadena al siguiente request
});
pm.test("Estado inicial OPEN", () =>
  pm.expect(pm.response.json().status).to.eql("OPEN"));
pm.test("La metadata NO contiene el binario del adjunto", () =>
  pm.expect(pm.response.json()).to.not.have.property("fileBytes"));
  
```

Variables de entorno (staging.postman_environment.json): baseUrl, jwt, ticketId, parentIncidentId. Permiten cambiar de entorno sin tocar los requests.

Automatización en CI (Newman): la colección se ejecuta en cada push como *gate*:

```

newman run TicketResolve.postman_collection.json \
  -e staging.postman_environment.json \
  --reporters cli,junit --reporter-junit-export results.xml
  
```

El reporte JUnit se integra al pipeline (mismo CI de Terraform en `.github/workflows/`). Si falla un test P0 → build rojo.

3.4 Pruebas de integración (caja gris)

Qué: que los datos **fluyan y persistan** correctamente entre servicios. Postman dispara el request y luego se verifica el efecto lateral en AWS.

Escenario de integración	Verificación
Crear ticket con adjunto	Item en DynamoDB (PK=TICKET#id) y objeto en S3; la metadata solo guarda la URL
Webhook duplicado	1 item padre + N items CHILD# colgando del mismo PK
Cerrar ticket	Mensaje encolado en SQS → notificacion-worker consume → publicación en SNS
Escalamiento	Evento de auditoría inmutable escrito en DynamoDB

Herramientas: Postman para disparar + **AWS CLI** para aserciones (`aws dynamodb get-item` , `aws s3 ls` , `aws sqs get-queue-attributes`), o **LocalStack** para correr la cadena AWS localmente sin costo.

3.5 Pruebas de UI

Qué: las 3 pantallas del producto (de los mockups de E1):

Pantalla	Pruebas
M1 — Portal de autoservicio	Validación de formulario (campos obligatorios, tamaño de adjunto), envío exitoso, mensaje de confirmación con ID
M2 — Dashboard de ingenieros	Orden por severidad y SLA, render del contador de SLA, refresco, estado vacío
M3 — Consola de diagnóstico	Timeline de eventos, panel de metadatos, expansión de los 499 duplicados consolidados

Herramienta: Cypress o Playwright. Ejemplo conceptual (Playwright):

```
test('Dashboard ordena P0 antes que P1 y P2 (US-03)', async ({ page }) => {
  await page.goto('/dashboard');
  const sev = await page.locator('[data-test=severity]').allInnerTexts();
  expect(sev).toEqual(['P0', 'P1', 'P2']); // orden por criticidad
});
```

Incluye chequeos de **accesibilidad** (axe), **responsive** y que el contador de SLA no muestre valores negativos.

3.6 Pruebas End-to-End (E2E)

Qué: un flujo de negocio completo cruzando UI + API + asíncrono:

E2E-1: María (UI) crea un ticket P1 → aparece en el dashboard de Carlos (UI) → Carlos lo resuelve (UI) → el sistema encola y envía email (API/SNS) → María ve el ticket como CLOSED y recibe la notificación.

Herramienta: Playwright para los pasos de UI + Postman/AWS CLI para verificar el lado asíncrono (que el email se publicó en SNS). Son **pocas** pruebas (lentas y frágiles) reservadas a los caminos críticos del negocio.

4. Tipos de prueba especializados

Transversales a los niveles anteriores:

Tipo	Propósito en TicketResolve	Cuándo / herramienta
Smoke	Funciones vitales: ¿se crea ticket?, ¿carga dashboard?, ¿responde webhook? Si falla → build rechazado.	Cada build · Newman (subset)
Sanity	Tras un fix puntual (ej. bug del hash), validar solo esa lógica.	Tras cada corrección
Regresión	Re-ejecutar toda la suite tras cambios grandes (ej. cambio de single-table design). Costosa → automatizada .	Antes de cada entrega · Newman + Cypress en CI
Mutation testing	Validar la <i>calidad de las pruebas</i> rompiendo el código a propósito.	Sobre lógica crítica · mutmut / Stryker

5. Requerimientos a probar

ID	Prioridad	Requerimiento	Criterio clave
US-01	P0	Reportar ticket con adjuntos	ID único, adjunto aislado en S3, respuesta < 2s
US-02	P0	Ingesta masiva de alertas por Webhook	Procesamiento asíncrono, incidentes P0
US-03	P0	Dashboard priorizado por severidad y SLA	Carga < 500ms
US-04	P1	Matriz de escalamiento automático cada 15 min	Escala si no hay <i>Acknowledge</i> en SLA
US-05	P1	Cerrar ticket con justificación + email asíncrono	Email al reportero vía SNS
US-06	P2	Reporte PDF mensual	Background, link presigned 24h
F-1	—	Motor de escalamiento jerárquico por guardias	Eleva ticket si no hay <i>Acknowledge</i> en SLA
F-2	—	Detección de duplicados por hashing	Padre + hijos en ventana de 5 min
F-3	—	Generador asíncrono de reportes post-mortem	No afecta carga transaccional

6. Selección de técnicas por requerimiento

Requerimiento	Técnica(s)	Por qué
US-01	Partición de equivalencia + Valores límite	Validar campos del formulario y bordes (tamaño de archivo).
US-02 / F-2	Valores límite + Caja blanca (branch)	Ventana de 5 min = borde temporal; rama "colisión vs nuevo padre".
US-03	Partición de equivalencia + Rendimiento	Ordenamiento + umbral <500ms.
US-04 / F-1	Tabla de decisión + Transición de estados + Caja blanca	Múltiples condiciones y cambios de estado.
US-05	Transición de estados + BDD	El cierre es una transición; el email asíncrono se especifica con Gherkin.
US-06 / F-3	Valores límite + BDD	Expiración de 24h = borde; flujo asíncrono narrado con BDD.

7. Técnicas de caja negra

7.1 Partición de equivalencia — Formulario de creación de ticket (US-01)

Campo	Clases válidas	Clases inválidas
Título	1–120 caracteres no vacíos	Vacío; solo espacios; > 120
Servicio afectado	Valor del catálogo (ERP, Pagos, Red...)	Fuera del catálogo; nulo
Severidad	P0 , P1 , P2	P3 ; p0 ; nulo
Descripción	1–2000 caracteres	> 2000
Adjunto (tipo)	png , jpg , log , pdf	exe , bat , sin extensión
Adjunto (tamaño)	1 byte – 25 MB	0 bytes; > 25 MB

7.2 Valores límite

(a) Ventana de deduplicación — F-2 (5 minutos):

Tiempo desde el evento padre	Resultado esperado
4 min 59 s	Se agrupa como hijo
5 min 00 s	Borde → inclusivo: aún es hijo
5 min 01 s	Se crea nuevo padre

(b) SLA de escalamiento — US-04 / F-1 (P0 = 15 min):

SLA sin Acknowledge	Resultado esperado
14 min 59 s	No escala
15 min 00 s	Escala a Nivel 2
29 min 59 s	Sigue en Nivel 2
30 min 00 s	Escala a Nivel 3 + notifica al Gerente

(c) Expiración de presigned URL — US-06 (24h):

Tiempo tras emisión	Resultado
23 h 59 m	Descarga OK (200)
24 h 00 m	Borde → expira
24 h 01 m	Denegada (403)

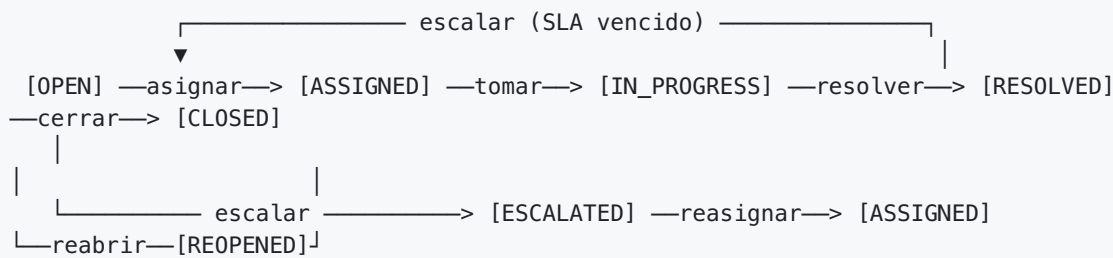
(d) Tamaño de adjunto — US-01 (25 MB): 0, 1 byte, 25 MB exactos, 25 MB + 1 byte.**7.3 Tabla de decisión — Motor de escalamiento (US-04 / F-1)**

Condiciones: C1 ¿SLA vencido?, C2 Nivel actual (N1/N2/N3), C3 ¿Guardia activa? **Acciones:** A1 escalar, A2 notificar gerente, A3 asignar a guardia, A4 no hacer nada.

Regla	C1	C2	C3	Acciones
R1	No	—	—	A4
R2	Sí	N1	Sí	A1 + A3
R3	Sí	N1	No	A1
R4	Sí	N2	Sí	A1 + A2 + A3
R5	Sí	N2	No	A1 + A2
R6	Sí	N3	—	A2 (nivel máximo; re-notifica, no escala)

R6 es un caso que las US no mencionaban: ¿qué pasa en N3 cuando el SLA vence de nuevo?
— hallazgo gracias a la técnica.

7.4 Transición de estados — Ciclo de vida del ticket (US-01, US-04, US-05)



Origen	Evento	Destino	¿Válida?
OPEN	Asignar	ASSIGNED	✓
OPEN	SLA vencido	ESCALATED	✓
ASSIGNED	Iniciar diagnóstico	IN_PROGRESS	✓
IN_PROGRESS	Resolver	RESOLVED	✓
RESOLVED	Cerrar	CLOSED	✓
RESOLVED	Reabrir	REOPENED	✓
CLOSED	Iniciar diagnóstico	IN_PROGRESS	✗ inválida — rechazar
CLOSED	Escalar	ESCALATED	✗ inválida — un cerrado no escala

8. Técnicas de caja blanca

8.1 Branch testing — webhook-ingesta (deduplicación, F-2)

```

def procesar_alerta(evento):
    h = hash(evento.componente + evento.mensaje) # S1
    padre = buscar_en_ventana(h, ventana=5*60) # S2
    if padre is not None: # B1
        crear_item_hijo(padre, evento) # S3 (rama verdadera)
    else:
        crear_ticket_padre(h, evento, severidad="P0") # S4 (rama falsa)
    return ok() # S5
  
```

- **Statement coverage:** un solo caso ejecuta S1,S2,S4,S5 pero **no** cubre la rama de duplicado. Insuficiente.
- **Branch coverage (objetivo):** 2 casos cubren ambas salidas de B1 → TC-B1a (sin padre → S4) y TC-B1b (mismo hash en 5 min → S3).

8.2 Branch testing — escalamiento-worker (conditional write, F-1)

```
def escalar(ticket):
    actual = leer(ticket.id)
    try:
        escribir_condicional(ticket.id, set_nivel=actual.nivel + 1,
                             condicion=(version == actual.version))    # B2: optimistic lock
    except ConflictException:
        log("escalamiento ya aplicado"); return    # rama de conflicto
    publicar_sqs(ticket.id)
```

- TC-B2a: una invocación → versión coincide → escribe y publica.
- TC-B2b: dos invocaciones concurrentes → la segunda recibe `ConflictException`, **no** re-escala ni duplica notificación (clave para el riesgo de race condition, §13).

9. Diseño de escenarios de prueba detallados

Formato del curso: *Título, Precondiciones, Pasos, Datos, Resultado esperado, Estado, Post-ejecución, Adjuntos*. Estados: **Passed / Failed / Skip-NA / Blocked**.

TC-01 — Crear ticket con adjunto válido (US-01) · API+UI · P0

Campo	Detalle
Precondiciones	Usuario con JWT válido; catálogo cargado; bucket S3 disponible
Datos	Título "ERP no carga"; Servicio "ERP"; Severidad P1; Adjunto <code>error.png</code> (2 MB)
Pasos	1. Llenar formulario. 2. Adjuntar <code>error.png</code> . 3. Click "Emitir Ticket".
Resultado esperado	HTTP 201 con <code>ticketId</code> único (TKT-####); estado <code>OPEN</code> ; respuesta < 2s; archivo en S3 (no en DynamoDB); adjunto no accesible sin presigned.
Herramienta	Postman (API) + Playwright (UI)
Estado	(Pendiente)

TC-02 — Rechazo de adjunto sobre el límite (US-01, valores límite) · API · P1

Campo	Detalle
Datos	Adjunto de 25 MB + 1 byte
Pasos	1. Formulario válido. 2. Adjuntar archivo de 25 MB + 1 byte. 3. Emitir.
Resultado esperado	HTTP 413 (o validación frontend); ticket no creado; mensaje claro de tamaño máximo.
Estado	(Pendiente)

TC-03 — Deduplicación en ráfaga de webhooks (US-02 / F-2) · Integración · P0

Campo	Detalle
Precondiciones	POST /api/v1/incidents activo; sin incidente previo con ese hash
Datos	500 payloads con mismo (componente="pagos", mensaje="connection refused") en 30s
Pasos	1. Disparar 500 POST concurrentes (k6/Newman). 2. Esperar drenaje de cola. 3. Consultar dashboard.
Resultado esperado	1 ticket padre P0 ; al expandir, 499 hijos ; 0 duplicados sueltos; 0 alertas perdidas.
Post-ejecución	Revisar Throttles / Errors de Lambda en CloudWatch.
Estado	(Pendiente)

TC-04 — Dashboard ordenado y bajo umbral (US-03) · UI+rendimiento · P0

Campo	Detalle
Precondiciones	Ingeniero "Carlos" con ≥ 3 tickets de distinta severidad
Pasos	1. Login como Carlos. 2. Abrir dashboard. 3. Medir p95 de 20 cargas.
Resultado esperado	Orden P0 → P1 → P2; latencia p95 < 500ms .
Estado	(Pendiente)

TC-05 — Escalamiento en el borde del SLA (US-04 / F-1) · Integración · P1

Campo	Detalle
Precondiciones	Matriz: P0 → N2 a 15 min, → N3 a 30 min; guardia Equipo A activa; reloj simulado
Pasos	1. Crear P0 sin Acknowledge. 2. Reloj a 14:59 → verificar. 3. Reloj a 15:00 → verificar.
Resultado esperado	14:59 sigue N1; 15:00 pasa a N2 , reasigna a Equipo A y publica notificación en SQS → email.
Post-ejecución	Verificar evento de auditoría inmutable en DynamoDB.
Estado	(Pendiente)

TC-06 — No doble escalamiento por concurrencia (F-1) · Caja blanca · P0-riesgo

Campo	Detalle
Pasos	1. Disparar 2 ejecuciones concurrentes del worker sobre el mismo ticket vencido.
Resultado esperado	Solo una aplica el escalamiento; la otra recibe <code>ConflictException</code> ; sin email duplicado.
Estado	(Pendiente)

TC-07 — Cierre con justificación y email asíncrono (US-05) · Integración · P1

Campo	Detalle
Precondiciones	Ticket en <code>IN_PROGRESS</code> asignado a Carlos; reportero María con email válido
Pasos	1. Agregar comentario técnico. 2. <code>RESOLVED</code> → <code>CLOSED</code> .
Resultado esperado	API responde sin esperar email; estado <code>CLOSED</code> ; SLA congelado; mensaje en SQS → <code>notificacion-worker</code> → email vía SNS.
Estado	<i>(Pendiente)</i>

TC-08 — Transición inválida: reactivar ticket cerrado (US-05) · API · P1

Campo	Detalle
Precondiciones	Ticket en <code>CLOSED</code>
Pasos	1. Intentar <code>CLOSED</code> → <code>IN_PROGRESS</code> vía API. 2. Simular escalamiento sobre él.
Resultado esperado	HTTP 409/422; permanece <code>CLOSED</code> ; el motor lo ignora.
Estado	<i>(Pendiente)</i>

TC-09 — Reporte PDF asíncrono con link expirable (US-06 / F-3) · E2E · P2

Campo	Detalle
Pasos	1. "Generar reporte". 2. Verificar respuesta inmediata. 3. Recibir email. 4. Descargar a 23h59 y reintentar a 24h01.
Resultado esperado	Respuesta inmediata "en proceso"; PDF en S3; email con presigned URL; descarga OK antes de 24h, 403 después .
Estado	<i>(Pendiente)</i>

TC-10 — Smoke test del build · Gate

Campo	Detalle
Pasos	1. Crear 1 ticket. 2. Cargar dashboard. 3. Enviar 1 webhook. (Newman subset)
Resultado esperado	Las 3 funciones vitales responden 2xx. Si falla → build rechazado .
Estado	<i>(Pendiente)</i>

TC-11 — Control de acceso por rol (RBAC / seguridad cloud) · API · P0-seguridad

Campo	Detalle
Precondiciones	Usuario con rol <code>Usuario Afectado</code> (no Admin) autenticado con JWT válido
Pasos	1. Intentar <code>PATCH</code> sobre la matriz de escalamiento (acción de Admin). 2. Intentar <code>DELETE</code> sobre un ticket ajeno.
Resultado esperado	HTTP 403 Forbidden en ambos; el JWT es válido pero el rol no autoriza; se registra el intento en el log de seguridad (CloudWatch).
Herramienta	Postman (variando el claim de rol en el JWT)
Estado	<i>(Pendiente)</i>

TC-12 — Resiliencia: reintentos y DLQ ante fallo del worker (alta disponibilidad) · Integración · P1

Campo	Detalle
Precondiciones	Cola SQS con redrive a DLQ configurada (<code>maxReceiveCount=3</code>); inyectar fallo transitorio en <code>notificacion-worker</code>
Pasos	1. Encolar mensaje de notificación. 2. Forzar que el worker falle 2 veces. 3. Dejar que el 3er intento tenga éxito.
Resultado esperado	El mensaje se reintenta automáticamente; al éxito se envía 1 solo email (idempotencia, sin duplicados); si superara los 3 intentos, el mensaje queda en DLQ sin perderse.
Post-ejecución	Verificar <code>ApproximateNumberOfMessagesVisible</code> en DLQ = 0 en el caso de éxito.
Estado	<i>(Pendiente)</i>

10. Especificación por comportamiento (BDD / Gherkin)

Técnica **Dado–Cuando–Entonces**: los criterios de aceptación se vuelven escenarios ejecutables (alineado con ATDD).

Feature: Deduplicación de alertas masivas (US-02 / F-2)

Scenario: Alertas idénticas dentro de la ventana se agrupan

Dado un sistema sin incidentes activos para el componente "pagos"

Cuando se reciben 500 webhooks con componente "pagos" y mensaje "connection refused" en 30 segundos

Entonces se crea 1 incidente padre con severidad "P0"

Y los otros 499 quedan como incidentes hijos del mismo padre

Y el dashboard muestra solo 1 ticket para ese componente

Scenario: Alerta fuera de la ventana de 5 minutos crea nuevo padre

Dado un incidente padre para "pagos / connection refused"

Cuando llega otra alerta idéntica 5 minutos y 1 segundo después

Entonces se crea un nuevo incidente padre

Y no se agrupa con el anterior

Feature: Escalamiento automático por SLA (US-04 / F-1)

Scenario: Escalar un P0 no atendido al cumplir su SLA

Dado un ticket "P0" en nivel 1 sin Acknowledge

Y una matriz que escala P0 a nivel 2 a los 15 minutos

Cuando transcurren 15 minutos sin Acknowledge

Entonces el ticket pasa a nivel 2

Y se asigna al equipo de guardia activo

Y se envía una notificación urgente por email al nuevo asignado

Scenario: No re-escalar un ticket ya cerrado

Dado un ticket en estado "CLOSED"

Cuando el motor de escalamiento ejecuta su ciclo

Entonces el ticket permanece "CLOSED"

Y no se genera ninguna notificación

Feature: Reporte mensual asíncrono (US-06 / F-3)

Scenario: El reporte se genera en segundo plano sin bloquear la API

Dado un Gerente de Operaciones autenticado

Cuando solicita el reporte mensual de abril

Entonces la API responde de inmediato con "en proceso"

Y posteriormente recibe un email con un enlace de descarga

Y el enlace deja de funcionar después de 24 horas

11. Pruebas no funcionales (rendimiento, estrés, seguridad)

11.1 Rendimiento y carga

Distinción del curso: *carga* evalúa la infraestructura ante la carga **esperada**; *estrés* evalúa los **límites** y más allá.

Escenario	Tipo	Carga	Métrica / umbral	Herramienta
Ingesta de webhooks (caso Datadog)	Carga	500 POST en 30s	0 alertas perdidas; 0 duplicados escapados; p95 estable	k6 / Artillery
Ingesta extrema	Estrés	5 000 POST en 30s	Hallar punto de quiebre; vigilar Throttles , ConcurrentExecutions , profundidad de SQS	k6 + CloudWatch
Dashboard concurrente (US-03)	Carga	50 ingenieros consultando	p95 < 500ms	k6
Generación de reportes (F-3)	Carga	10 reportes en paralelo	La API transaccional no se degrada	k6

Script de carga (k6) — concepto para el caso Datadog:

```
import http from 'k6/http';
import { check } from 'k6';

export const options = {
  scenarios: {
    webhook_burst: { executor: 'per-vu-iterations', vus: 500, iterations: 1, maxDuration: '30s' },
  },
  thresholds: {
    http_req_duration: ['p(95)<500'], // latencia
    http_req_failed: ['rate<0.01'], // < 1% de error
  },
};

export default function () {
  const payload = JSON.stringify({ componente: 'pagos', mensaje: 'connection refused' });
  const res = http.post(`${__ENV.BASE_URL}/api/v1/incidents`, payload, {
    headers: { 'Content-Type': 'application/json' }
  });
  check(res, { 'aceptado (2xx)': (r) => r.status >= 200 && r.status < 300 });
}
```

Métricas a capturar: throughput (req/s), p95/p99 de latencia, tasa de error, y del lado AWS: Throttles , ConcurrentExecutions , ApproximateNumberOfMessagesVisible (SQS), consumo de WCU/RCU en DynamoDB.

11.2 Estrés — qué se busca encontrar

- **Punto de quiebre:** ¿a cuántos webhooks/seg empieza Lambda a hacer *throttling*?
- **Degradación elegante:** al saturar, ¿se pierden alertas o se acumulan en SQS y se procesan luego? (lo esperado es lo segundo, gracias al desacople).
- **Recuperación:** tras la ráfaga, ¿la cola se drena y el sistema vuelve a la normalidad?

11.3 Seguridad

Prueba	Técnica / herramienta	Resultado esperado
JWT expirado/inválido en la API	Caja gris · Postman	HTTP 401; sin acceso a datos
Acceso directo a adjunto S3 sin presigned	DAST · Postman/curl	HTTP 403 (aislamiento de adjuntos, US-01)
Expiración de presigned URL de reporte	Valores límite (24h) · Postman	403 tras vencer
Permisos IAM por Lambda	Análisis estático · tfsec / Checkov	Least-privilege: webhook-ingesta no puede leer el bucket de reportes
Inyección/XSS en título y descripción	DAST · OWASP ZAP	Sanitización; sin XSS al renderizar en dashboard
Dependencias vulnerables	SCA · npm audit / Snyk	0 vulnerabilidades críticas

11.4 Calidad de las pruebas — Mutation testing

Para validar que la suite **realmente detecta fallos**, se introducen mutantes en la lógica crítica (ej. `5*60 → 4*60` en la ventana de dedup, o `>= → >` en el umbral de SLA). Si ninguna prueba falla, hay un hueco en los escenarios de valores límite (§7.2). Herramienta: `mutmut` (Python) o `Stryker` (JS).

12. Pruebas de aceptación de usuario (UAT)

Objetivo: confirmar que el sistema **sirve para el negocio** con datos reales, validado por los actores dueños del proceso — no por QA.

12.1 Participantes y sesiones

Sesión	Actor que valida	Qué valida
UAT-1	Administrador de Plataforma	Configurar matriz de escalamiento y calendario de guardias, y ver que un P0 real escala como se configuró
UAT-2	Ingeniero de Soporte	Que el dashboard prioriza bien y que la consola de diagnóstico consolida los duplicados de forma útil
UAT-3	Gerente de Operaciones	Solicitar el reporte mensual y confirmar que las métricas de SLA reflejan la realidad
UAT-4	Usuario Afectado	Reportar un incidente desde el portal y recibir el email de cierre

12.2 Escenarios de aceptación (lenguaje de negocio)

ID	Como...	Quiero...	Acepto si...
UAT-A1	Administrador	configurar que un P0 escale en 15 min	un P0 de prueba sin atender escala automáticamente y notifica al siguiente nivel
UAT-A2	Ingeniero	ver mis tickets por criticidad	el P0 aparece arriba y la consola muestra los 499 duplicados agrupados en 1
UAT-A3	Gerente	un reporte mensual sin esperar en pantalla	recibo el PDF por correo con SLA cumplido por severidad
UAT-A4	Usuario	reportar y enterarme del cierre	recibo el ID al crear y un email cuando se resuelve

12.3 Criterio de cierre de UAT (sign-off)

- Todos los escenarios UAT-A* aceptados por su actor responsable.
- Sin defectos de severidad alta abiertos.
- **Acta de aceptación** firmada por el Administrador y el Gerente → habilita la defensa final.

13. Gestión de riesgos

Cada riesgo se evalúa por **probabilidad x impacto** para priorizar el esfuerzo de prueba.

ID	Riesgo	Prob.	Impacto	Exposición	Mitigación de prueba
R-01	Race condition en escalamiento (invocaciones solapadas)	Media	Alto	Alta	TC-06 concurrencia sobre conditional write (§8.2)
R-02	Falsos negativos en hash → dashboard saturado	Media	Alto	Alta	Valores límite ventana 5 min (§7.2) + branch coverage
R-03	Throttling de Lambda en ráfaga de webhooks	Alta	Medio	Alta	Prueba de estrés (§11.2) con monitoreo de <code>Throttles</code>
R-04	Presigned URL no expira → fuga de reportes	Baja	Alto	Media	Prueba de seguridad de expiración (24h)
R-05	Transición inválida reactiva un ticket cerrado	Baja	Medio	Baja	TC-08 transición de estados
R-06	Ambigüedad en SLAs por severidad	Alta	Bajo	Media	Documentados como supuestos + confirmados en UAT
R-07	Fuga de datos sensibles en el entorno de pruebas	Baja	Alto	Media	Data masking + datos sintéticos (§15); nunca usar datos reales de incidentes en staging
R-08	Inestabilidad del proveedor cloud durante pruebas de carga	Media	Medio	Media	Pruebas de carga escalonadas; coordinar ventana con el equipo de infra (curso IaC)

Los riesgos de **exposición Alta (R-01, R-02, R-03)** concentran las pruebas más rigurosas (concurrencia, branch coverage y estrés), coherente con la lógica de costo de defectos de §1.2.

14. Matriz de trazabilidad (RTM)

La trazabilidad requerimiento → caso de prueba se gestiona en **Jira Software + plugin Xray**, que vincula cada requerimiento con sus casos, ejecuciones y defectos, y produce los reportes de cobertura para el criterio de salida (§2.4).

Requerimiento / Atributo	Técnica(s)	Nivel / herramienta	Escenarios	BDD	No funcional	Riesgo
US-01	Partición / Valores límite	API (Postman) + UI	TC-01, TC-02	—	Seguridad (aislamiento)	—
US-02 / F-2	Valores límite / Branch	Integración + Carga	TC-03	dedup	Carga + estrés	R-02, R-03
US-03	Partición / Rendimiento	UI + k6	TC-04	—	Carga (<500ms)	—
US-04 / F-1	Tabla decisión / Estados / Branch	Integración + Caja blanca	TC-05, TC-06	escalamiento	—	R-01
US-05	Transición de estados	Integración + API	TC-07, TC-08	cierre	—	R-05
US-06 / F-3	Valores límite / BDD	E2E	TC-09	reporte	Seguridad presigned	R-04
Seguridad cloud (RBAC)	Partición (roles)	API (Postman)	TC-11	—	Seguridad	R-07
Alta disponibilidad / resiliencia	—	Integración (SQS/DLQ)	TC-12	—	Resiliencia	R-08
Build gate	Smoke	Newman CI	TC-10	—	—	—

15. Supuestos y datos de prueba

Valores no definidos en las US (a confirmar en UAT — evita escenarios mal diseñados):

Parámetro	Valor asumido	Confirma
SLA P0 / escalamiento	N2 a 15 min, N3 a 30 min	Administrador
SLA P1 / P2	P1 ≈ 2h, P2 ≈ 24h	Administrador
Ventana de deduplicación	5 min (borde inclusivo)	Diseño F-2
Tamaño máximo de adjunto	25 MB	Desarrollo
Vigencia de presigned URL	24 h	Diseño US-06
Umbral dashboard	p95 < 500 ms	Criterio US-03
Tiempo de respuesta creación	< 2 s	Narrativa US-01

Datos semilla: 1 ingeniero ("Carlos") con tickets P0/P1/P2; 1 reportero ("María"); matriz de escalamiento de 3 niveles; calendario L-V Equipo A / fines de semana Equipo B; 500 payloads idénticos para deduplicación; colección Postman + entorno staging versionados en el repo.

Gestión de datos de prueba (seguridad): en staging **nunca** se usan datos reales de incidentes. Se generan **datos sintéticos** y, si se parte de un export real, se aplica **data masking** (anonimizar emails, nombres y descripciones) para mitigar el riesgo R-07. Esto se alinea con el cifrado en reposo (SSE) de la arquitectura.

16. Clasificación de severidad y prioridad de defectos

Estandariza la comunicación entre QA y Desarrollo. **Severidad** = impacto técnico; **Prioridad** = urgencia de corrección (no siempre coinciden).

Nivel	Definición	Ejemplo en TicketResolve	Prioridad típica
S1 — Bloqueante	Detiene el ciclo de pruebas o impide usar el sistema	La tabla DynamoDB no responde; la API devuelve 5xx en todo	Inmediata
S2 — Crítico	Funcionalidad vital dañada, sin <i>workaround</i>	El motor de escalamiento no escala P0; la deduplicación deja pasar 500 tickets	Alta
S3 — Mayor	Funcionalidad afectada pero con alternativa manual	El dashboard no ordena por SLA pero sí se puede filtrar a mano	Media
S4 — Menor	Cosmético / no afecta la operación	Texto desalineado en la consola; typo en un label	Baja

Relación con el criterio de salida (§2.4): se exige **0 defectos S1/S2 abiertos** para certificar.

17. Cronograma de ejecución

Plan de 4 semanas alineado con el modelo *Shift-Left* (§1.1): la calidad arranca desde el análisis, no al final.

Semana	Foco	Actividades	Niveles / herramientas
1	Fundamentos y código	Revisión estática de requerimientos, diseño de casos (este documento), pruebas unitarias del core (hash, SLA)	Unitarias · pytest/jest
2	Integración y funcionalidad	API y persistencia (Lambda→DynamoDB→S3); ~60% de los casos funcionales	API/Integración · Postman/Newman + AWS CLI
3	Robustez y seguridad	Carga y estrés (500/5000 webhooks), seguridad (RBAC, presigned, tfsec/ZAP), regresión de bugs corregidos	No funcional · k6, OWASP ZAP
4	Aceptación y certificación	UI/E2E, UAT con Admin y Gerente, validación de KPIs y reportes, acta de salida firmada	UI/E2E · Cypress/Playwright + UAT

El cronograma encaja con el calendario del proyecto (defensa final 18 jun 2026); la Semana 4 cierra con el *sign-off* de UAT (§12.3).

18. Anexos — artefactos de prueba ejecutables

Estos artefactos materializan los niveles descritos en §3 y §11. Pueden copiarse a archivos y ejecutarse tal cual.

Anexo A — Colección Postman (API/Servicios)

Guardar como `TicketResolve.postman_collection.json` e importar en Postman. Las aserciones (`pm.test`) corren al ejecutar la colección o vía Newman en CI.


```

{
  "info": {
    "name": "TicketResolve API (v1)",
    "schema": "https://schema.getpostman.com/json/collection/v2.1.0/collection.json"
  },
  "variable": [
    { "key": "baseUrl", "value": "https://staging.ticketresolve.example" },
    { "key": "jwt", "value": "" },
    { "key": "ticketId", "value": "" }
  ],
  "item": [
    {
      "name": "01 · Tickets",
      "item": [
        {
          "name": "POST Crear ticket",
          "request": {
            "method": "POST",
            "header": [
              { "key": "Authorization", "value": "Bearer {{jwt}}" },
              { "key": "Content-Type", "value": "application/json" }
            ],
            "url": "{{baseUrl}}/api/v1/tickets",
            "body": {
              "mode": "raw",
              "raw": "{\n  \"titulo\": \"ERP no carga\",\n  \"servicio\": \"ERP\",\n  \"severidad\": \"P1\",\n  \"descripcion\": \"No se puede facturar\"\n}"
            }
          },
          "event": [
            {
              "listen": "test",
              "script": {
                "exec": [
                  "pm.test('201 Created', () => pm.response.to.have.status(201));",
                  "pm.test('Responde en < 2s (US-01)', () => pm.expect(pm.response.responseTime).to.be.below(2000));",
                  "pm.test('ID con formato TKT-###', () => {",
                    "  const b = pm.response.json();",
                    "  pm.expect(b.ticketId).to.match(/^TKT-\\d+$/);",
                    "  pm.collectionVariables.set('ticketId', b.ticketId);",
                  "});",
                  "pm.test('Estado inicial OPEN', () => pm.expect(pm.response.json().status).to.eql('OPEN'));"
                ]
              }
            }
          ]
        },
        {
          "name": "GET Listar por ingeniero",
          "request": {
            "method": "GET",
            "header": [ { "key": "Authorization", "value": "Bearer {{jwt}}" } ],
            "url": "{{baseUrl}}/api/v1/tickets?engineer=carlos"
          },
          "event": [
            {
              "listen": "test",
              "script": {
                "exec": [

```

```

        "pm.test('200 OK', () => pm.response.to.have.status(200));",
        "pm.test('Ordenado por severidad (P0 primero)', () => {",
        "    const sev = pm.response.json().items.map(t => t.severidad);",
        "    const orden = { P0: 0, P1: 1, P2: 2 };",
        "    for (let i = 1; i < sev.length; i++)",
        "        pm.expect(orden[sev[i-1]]).to.be.at.most(orden[sev[i]]);",
        "});"
    ]
}
}
],
{
    "name": "POST Crear ticket SIN token (negativo)",
    "request": {
        "method": "POST",
        "header": [ { "key": "Content-Type", "value": "application/json" } ],
        "url": "{{baseUrl}}/api/v1/tickets",
        "body": { "mode": "raw", "raw": "{ \"titulo\": \"x\", \"servicio\": \"ERP\", \"severidad\": \"P2\" }" }
    },
    "event": [
        {
            "listen": "test",
            "script": { "exec": [ "pm.test('401 sin JWT', () => pm.response.to.have.status(401));" ] }
        }
    ]
}
],
{
    "name": "02 · Incidents (Webhook)",
    "item": [
        {
            "name": "POST Alerta (dedup)",
            "request": {
                "method": "POST",
                "header": [ { "key": "Content-Type", "value": "application/json" } ],
                "url": "{{baseUrl}}/api/v1/incidents",
                "body": { "mode": "raw", "raw": "{ \"componente\": \"pagos\", \"mensaje\": \"connection refused\" }" }
            },
            "event": [
                {
                    "listen": "test",
                    "script": { "exec": [ "pm.test('Aceptado (2xx)', () => pm.expect(pm.response.code).to.be.within(200, 299));" ] }
                }
            ]
        }
    ]
}
],
{
    "name": "99 · Negativos / seguridad",
    "item": [
        {
            "name": "PATCH CLOSED -> IN_PROGRESS (transición inválida, TC-08)",
            "request": {
                "method": "PATCH",
                "header": [

```

```

        { "key": "Authorization", "value": "Bearer {{jwt}}" },
        { "key": "Content-Type", "value": "application/json" }
      ],
      "url": "{{baseUrl}}/api/v1/tickets/{{ticketId}}",
      "body": { "mode": "raw", "raw": "{ \"status\": \"IN_PROGRESS\" }" }
    },
    "event": [
      {
        "listen": "test",
        "script": { "exec": [ "pm.test('409 transición inválida', () =>
pm.response.to.have.status(409));" ] }
      }
    ]
  }
]
}
]
}
}

```

Ejecución en CI con Newman:

```

newman run TicketResolve.postman_collection.json \
  -e staging.postman_environment.json \
  --reporters cli,junit --reporter-junit-export results.xml

```

Anexo B — Script de carga k6 (no funcional · TC-03)

Guardar como `webhook-burst.js` y ejecutar con `k6 run -e BASE_URL=https://staging.ticketresolve.example webhook-burst.js`.

```

import http from 'k6/http';
import { check } from 'k6';

// Simula la "tormenta de tickets": 500 alertas idénticas en 30s (caso Datadog)
export const options = {
  scenarios: {
    webhook_burst: {
      executor: 'per-vu-iterations',
      vus: 500, iterations: 1, maxDuration: '30s',
    },
  },
  thresholds: {
    http_req_duration: ['p(95)<500'], // latencia p95 < 500ms
    http_req_failed: ['rate<0.01'], // < 1% de error
  },
};

export default function () {
  const payload = JSON.stringify({ componente: 'pagos', mensaje: 'connection refused' });
  const res = http.post(`${__ENV.BASE_URL}/api/v1/incidents`, payload, {
    headers: { 'Content-Type': 'application/json' },
  });
  check(res, { 'aceptado (2xx)': (r) => r.status >= 200 && r.status < 300 });
}

```

Tras la corrida, verificar en CloudWatch que `Throttles = 0` y que la cola SQS se drenó: el dashboard debe mostrar **1 padre + 499 hijos**, no 500 tickets.

Anexo C — Prueba de UI con Playwright (TC-04)

Guardar como `dashboard.spec.ts` y ejecutar con `npx playwright test`.

```
import { test, expect } from '@playwright/test';

test('Dashboard ordena por severidad y carga < 500ms (US-03)', async ({ page }) => {
  const inicio = Date.now();
  await page.goto('/dashboard');
  await page.locator('[data-test=ticket-row]').first().waitFor();
  const latencia = Date.now() - inicio;

  // Orden por criticidad
  const sev = await page.locator('[data-test=severity]').allInnerTexts();
  const orden = { P0: 0, P1: 1, P2: 2 } as Record<string, number>;
  for (let i = 1; i < sev.length; i++) {
    expect(orden[sev[i - 1]]).toBeLessThanOrEqual(orden[sev[i]]);
  }

  // Umbral de rendimiento percibido
  expect(latencia).toBeLessThan(500);

  // El contador de SLA nunca es negativo
  const slas = await page.locator('[data-test=sla-restante]').allInnerTexts();
  for (const s of slas) expect(s).not.toContain('-');
});
```

Cómo se usó la guía del curso: este plan aterriza los conceptos de la *Mega-Guía* — estructura ISTQB del plan (§2), niveles de prueba con herramientas reales (§3), costo de defectos y SDLC (§1), las 4 técnicas de caja negra (§7), statement/branch de caja blanca (§8), formato profesional de escenarios con estados Passed/Failed/Skip/Blocked (§9), BDD/Gherkin y ATDD (§10), pruebas de carga/estrés/seguridad (§11), UAT con sign-off (§12), gestión de riesgos por exposición (§13), clasificación de defectos S1–S4 (§16) y cronograma Shift-Left (§17) — sobre los requerimientos reales de TicketResolve.